

CYBER BOX: PROJECT DOCUMENTATION & TECHNICAL REPORT

1. EXECUTIVE SUMMARY

Cyber Box is a next-generation interactive fitness application that transforms a standard web browser into an AI-powered boxing gym. By leveraging the latest advancements in **Computer Vision (CV)** and **Machine Learning (ML)** directly in the client-side browser, Cyber Box eliminates the need for expensive consoles, VR headsets, or wearable sensors.

The system uses a standard webcam to track the user's skeletal mechanics in real-time (30+ FPS). It maps the player's physical movements—ducks, slips, weaves—into a 3D cyberpunk environment where they must dodge incoming attacks from a virtual opponent.

Unlike traditional "exergames" that rely on simple motion triggers, Cyber Box operates on a **"Form First" philosophy**. It does not just detect movement; it evaluates *technique*. Using a custom-trained K-Nearest Neighbors (KNN) classifier, the system learns the user's specific biomechanics to distinguish between a "Lazy Lean" and a "Proper Slip," gamifying the acquisition of real-world boxing reflexes.

2. TECHNICAL ARCHITECTURE

The project is built on a modern React ecosystem, optimized for high-performance rendering and low-latency inference.

2.1. The Tech Stack

- **Frontend Framework:** React 18 (TypeScript) via Vite.
- **Computer Vision:** Google MediaPipe Pose (Blazepose model).
- **Machine Learning:** TensorFlow.js + KNN Classifier.
- **3D Graphics:** Three.js via React Three Fiber (R3F) & Drei.
- **Styling:** Tailwind CSS.
- **Generative AI:** Google Gemini 1.5 Flash API (for Coaching).

2.2. The "Ref-First" Architecture

A critical architectural decision was the move away from React's standard Declarative State for the game loop.

- **Problem:** React's state updates are asynchronous and trigger re-renders. In a 60 FPS computer vision loop, waiting for a React state update introduces lag and "stale closure" bugs.
 - **Solution:** We implemented a **Ref-First Architecture**.
 - **Logic State:** Critical game data (`gameState`, `timeLeft`, `activePunch`) is stored in `useRef` hooks. This allows the game loop to read/write data synchronously without triggering re-renders.
 - **UI State:** A parallel `useState` layer is updated *only* when visual feedback is required (e.g., score change), decoupling the high-speed logic loop from the slower rendering cycle.
-

3. CORE FEATURES & MECHANICS

3.1. Vision-Based Controls (The "Eyes")

The system tracks 33 distinct skeletal landmarks. However, raw coordinates are noisy and scale-dependent. We created a **Feature Extractor** module that converts raw (x , y , z) coordinates into normalized **Biomechanical Features**:

- **Head Displacement (DX/DY):** Movement relative to the calibrated "Neutral" spine center.
- **Spine Angle:** The calculated tilt of the torso.
- **Hip Drop:** Vertical compression (squat depth).
- **Velocity Vectors:** Differentiating position over time to detect fast movements vs. slow drifts.

3.2. Personalized Machine Learning (The "Brain")

Cyber Box does not use a generic, pre-baked AI model. Every session begins with a rapid **"Training Montage"**:

1. **Data Collection:** The user is prompted to hold specific poses: "Neutral", "Slip Left", "Slip Right", "Duck".
2. **Few-Shot Learning:** We capture ~50 frames per pose.
3. **Instant Training:** A **K-Nearest Neighbors (KNN)** classifier stores these feature vectors in memory.
4. **Inference:** During gameplay, the live pose is compared against this personal dataset. This ensures the game adapts to *any* body type, height, or distance from the camera instantly.

3.3. The "Oval Theory" Hit Detection (The "Judge")

Hitboxes in Cyber Box are not simple rectangles. We implement **Elliptical Threat Zones** based on the punch type:

- **Straights (Horizontal Oval):** Requires lateral movement. If the player ducks vertically, they might still be in the danger zone.

- **Hooks (Vertical Oval):** Requires vertical drops (ducking). Lateral movement is penalized as it keeps the head on the same plane as the hook.

Granular Scoring System:

- **PERFECT (300pts):** Clean dodge, correct technique (e.g., Slipping a Straight).
- **LUCKY (100pts):** Dodged, but bad form (e.g., Slipping a Hook).
- **RISKY (50pts):** Barely cleared the hit zone.
- **TOO_FAR:** Moved excessively, breaking stance/balance.
- **CAMPING:** Penalized for sitting in a dodge position without returning to center.

3.4. Adaptive Opponent AI

The virtual opponent utilizes a **Reinforcement Learning** strategy (Q-Learning Concept):

- **Confidence Table:** The AI tracks its success rate for each punch type (Left Straight, Right Hook, etc.).
- **Exploitation:** If the player repeatedly fails to dodge Left Hooks, the AI increases the probability of throwing Left Hooks.
- **Result:** The game naturally evolves to target the player's specific weaknesses.

3.5. Real-Time Gemini Coach

After a match, the `FightLog` (a JSON record of every timestamp, punch thrown, and player reaction) is sent to the **Gemini 1.5 Flash API**.

- **Prompt Engineering:** We inject the raw data into a prompt acting as a "Professional Boxing Coach."
- **Output:** The user receives specific, actionable advice (e.g., *"You're leaning forward too much on the Right Slips, which makes your recovery slow. Keep your posture upright."*)

4. CHALLENGES & ENGINEERING SOLUTIONS

Challenge 1: The "Death Loop" (React State vs. MediaPipe)

- **Problem:** Initially, we included game state variables in the `useEffect` dependency array for the MediaPipe loop. This caused the Camera and AI Model to initialize, run one frame, detect a state change, tear down, and re-initialize. This resulted in a flickering 1 FPS experience.
- **Solution:** We moved to an **Empty Dependency Array** `[]` for the main loop. We used `useRef` to access the changing state variables *inside* the loop without triggering a re-run of the effect hook.

Challenge 2: The "Timing Collision" Bug

- **Problem:** The game simply stopped spawning punches after the calibration phase.

- **Root Cause:** The `scheduleNextPunch` function was dependent on `timeLeft` (the round timer). Every second, the clock ticked, forcing React to recreate the scheduler function. The cleanup function `clearTimeout` would run, killing the punch that was waiting to be thrown.
- **Solution:** We created a **Silent Timer** using `useRef`. The scheduler no longer depends on the visual clock state, allowing punches to exist independently of the UI updates.

Challenge 3: Calibration Drift

- **Problem:** If a player calibrated while sitting and then stood up, the game would think they were constantly "Ducking" or out of position.
- **Solution:** We implemented **Dynamic Normalization**. The `FeatureExtractor` captures a "Baseline" (Nose Base, Shoulder Width) at the exact moment the game transitions to `PLAYING`. All subsequent movements are calculated as ratios relative to this baseline, making the game robust to position shifts.

Challenge 4: Loop Thrashing (State Transition)

- **Problem:** When calibration finished, the game would try to transition to `PLAYING` multiple times per second, causing a crash.
- **Solution:** We implemented a `isTransitioningRef` Guard Clause. This boolean lock ensures that the "Finish Calibration" logic executes exactly once, even if the vision loop calls it 60 times a second.

5. FUTURE ROADMAP

1. **Opponent Visuals:** Replace the abstract 3D models with rigged humanoid animations (GLB) to provide visual cues (telegraphing punches).
2. **Audio Engine:** Add spatial audio cues for punches whizzing past the ears to enhance immersion.
3. **Multiplayer:** Implement WebRTC to allow two players to shadow box "against" each other by syncing their punch/dodge data in real-time.

6. CONCLUSION

Cyber Box represents a successful convergence of modern web technologies. By optimizing the interaction between React's render cycle and TensorFlow's inference loop, we achieved a seamless, 60 FPS experience on standard hardware. It proves that advanced biomechanical analysis and immersive gaming need not be mutually exclusive—nor expensive. It is a gym in a browser tab.